

Mohawk Nexus: A Unified Multi-Language Network Stack for High-Performance Kernel-Bypass Packet Processing

Academic Review

Source Repository: <https://github.com/rwilliamspbg-ops/Mohawk-Nexus>

May 26, 2026

Abstract

Modern network stacks face a fundamental tension between performance and safety. Kernel-bypass technologies such as AF_XDP enable user-space packet processing at near-line-rate speeds, yet constructing a complete production-ready stack requires balancing low-level datapath optimization with high-level orchestration, cryptographic assurance, and formal verification. *Mohawk Nexus* is a unified, open-source network stack that addresses this tension through a deliberate architectural split: a **Go control plane** manages orchestration, lifecycle, configuration, and observability, while a **Rust datapath** executes the hot packet-forwarding path integrated with the Linux kernel’s AF_XDP subsystem. A canonical *bridge contract* governs the cross-language boundary, ensuring that control-plane state is batched and telemetry is aggregated without per-packet language transitions. The stack further incorporates multi-path spraying, hybrid post-quantum cryptography, and formal verification assets traceable to protocol semantics. This paper presents the architecture, implementation methodology, performance characteristics, and integration strategy of Mohawk Nexus, positioning it as a reference design for sovereign, high-performance, and formally traceable networking infrastructure.

1 Introduction

1.1 Motivation

The Linux kernel networking stack, while robust and general-purpose, introduces significant overhead for workloads that require custom L2–L3 processing, specialized routing, or application-specific optimization [2]. Kernel-bypass solutions such as DPDK achieve high throughput by monopolizing CPU cores and NIC resources, but at the cost of ecosystem isolation, driver maintenance burden, and operational complexity [9]. AF_XDP (Address Family Express Data Path) offers a compelling middle ground: it leverages existing kernel NIC drivers while providing a fast-path that redirects selected traffic to user-space sockets with zero-copy semantics [3].

However, AF_XDP alone does not constitute a complete network stack. Production deployments require control-plane orchestration, configuration management, telemetry, cryptographic session handling, and—increasingly—formal assurance that the implementation matches a specified protocol semantics. Building all of these concerns in a single language and runtime is challenging. Systems languages such as Rust excel at zero-allocation datapath programming, while Go offers superior ecosystem support for service orchestration, tooling, and developer velocity.

1.2 Contributions

Mohawk Nexus makes the following contributions:

1. **Architectural Unification:** It demonstrates a clean separation of concerns where Rust owns the packet hot-path and Go owns the control plane, connected via a statically defined bridge contract rather than ad-hoc inter-process communication.
2. **Performance with Safety:** The Rust datapath achieves 2.4–2.5 Mpps single-core forwarding and up to 75 GB/s effective throughput while maintaining Rust’s memory-safety guarantees and integrating formal verification artifacts.
3. **Cryptographic Sovereignty:** It implements hybrid post-quantum cryptography (PQC-ready) with in-place encryption/decryption to minimize allocations, achieving sub-microsecond crypto latency.
4. **Formal Traceability:** Protocol semantics, security assumptions, and verification claims are centralized in a dedicated proto repository and mapped onto implementation seams through a unified traceability matrix.
5. **Reproducible Integration Engineering:** The workspace provides pinned toolchains, containerized stress harnesses, and CI-resilient verification targets that remain green even when component sub-repositories are in flight.

2 Background and Related Work

2.1 AF_XDP and Kernel Fast-Paths

AF_XDP is not a kernel bypass in the DPDK sense; rather, it is an *in-kernel fast-path* that cooperates with the existing networking stack [3]. An eBPF program attached to the XDP hook executes at the earliest point in the NIC driver, deciding whether to pass packets to the normal kernel stack (XDP_PASS), drop them (XDP_DROP), or redirect them to an AF_XDP socket (XDP_REDIRECT). The socket operates on a shared memory region called UMEM, divided into equal-sized frames, with four single-producer/single-consumer ring buffers (RX, TX, FILL, COMPLETION) transferring frame descriptors between kernel and user-space without per-packet memory allocation [4].

When the NIC and driver support it, zero-copy mode allows DMA directly into the UMEM region, eliminating copy overhead entirely [5]. This model has been shown to achieve speeds competitive with DPDK for many use cases while retaining the ability to fall back to normal kernel networking for unselected traffic [9].

2.2 Multi-Language Network Stacks

Prior work in high-performance networking has often been monolithic: DPDK applications are typically written in C/C++, while newer frameworks such as Calico’s eBPF datapath or Cilium use Go for control and C for kernel code. The trend toward Rust in datapath programming is driven by its zero-cost abstractions, memory safety, and increasingly mature async ecosystem. Mohawk Nexus extends this trend by explicitly defining the control/datapath boundary and governing it with a contract rather than implicit conventions.

2.3 Formal Methods in Networking

Formal verification of network protocols has traditionally been applied to control-plane protocols (e.g., BGP route convergence) or transport semantics (e.g., TCP). Mohawk Nexus brings formal artifacts closer to the datapath by maintaining a **Sovereign-Mohawk-Proto** repository that houses protocol definitions, security models, and traceability matrices, with explicit mappings to Go and Rust implementation files.

3 Architecture

3.1 Design Philosophy

The overarching design rule of Mohawk Nexus is: *keep packet-rate work out of the Go/Rust boundary*. The datapath remains entirely within Rust, while Go sends coarse-grained configuration, route updates, and session state in batches. Rust returns aggregated telemetry (counters, health states, queue depths) rather than per-packet events. This batching philosophy eliminates language-boundary overhead from the critical path.

3.2 Workspace Layout

The repository is organized as a unified workspace with the following top-level structure [1]:

Listing 1: Mohawk Nexus Workspace Layout

```
Mohawk-Nexus/
|-- SMIP-MWP/                # Go control plane
|   |-- cmd/mohawk-node/      # Runtime entrypoint
|   |-- internal/bridge/      # Bridge contract adapter
|   |-- internal/datapath/afxdp/ # AF_XDP host integration
|   |-- internal/crypto/      # Crypto bindings
|-- SMIP-MWP-Rust/           # Rust datapath
|   |-- datapath/src/lib.rs   # Forwarding engine
|   |-- wire/src/lib.rs       # Packet format
|   |-- cli/src/main.rs       # Runtime entrypoint
|   |-- cli/src/bridge.rs     # Bridge adapter
|-- Sovereign-Mohawk-Proto/   # Protocol & verification source
|   |-- FORMAL_TRACEABILITY_MATRIX.md
|   |-- docs/{SECURITY,ARCHITECTURE,PERFORMANCE,OPERATIONS}.md
|-- bridge/                   # Canonical bridge schema & manifest
|-- scripts/                  # Workspace-level generation & validation
|-- go.work                   # Root Go workspace
|-- .github/workflows/ci.yml  # CI verification
```

3.3 Component Responsibilities

Go Control Plane (SMIP-MWP). The Go layer is the source of truth for runtime configuration. It handles:

- AF_XDP host preparation (hugepages, IRQ affinity, NIC queue binding)
- Route and session lifecycle management
- Metrics endpoint and telemetry aggregation
- Benchmark harnesses, profiling (pprof), and stress-test orchestration
- Containerized deployment (Docker, docker-compose)

Rust Datapath (SMIP-MWP-Rust). The Rust layer is the source of truth for hot-path execution. It handles:

- AF_XDP socket and ring ownership
- Zero-copy packet representation and forwarding
- Multi-path spraying (MRC) decisions
- In-place cryptographic operations
- Session state transitions at packet rate

Sovereign Protocol Layer (Sovereign-Mohawk-Proto). This repository serves as the semantic anchor:

- Transport and federation requirements
- Formal verification artifacts (Lean 4)
- Security posture, identity, and compliance constraints
- Higher-level control-plane behavior specifications

3.4 The Bridge Contract

The Go-Rust bridge contract is the central integration seam. It defines:

Control Request Shape:

Listing 2: Control Request Structure

```
ControlRequest {
  runtime_config
  route_updates[]
  session_updates[]
  feature_flags
}
```

Telemetry Response Shape:

Listing 3: Telemetry Response Structure

```
TelemetryResponse {
  health_state
  forwarder_stats
  queue_stats
  timestamp
}
```

Key ownership rules [6]:

- Go decides *when* to apply configuration; Rust decides *how* to execute packets.
- Go can request start, stop, or reload; Rust owns packet-level state transitions.
- Formal protocol claims live in **Sovereign-Mohawk-Proto** and must map onto this boundary.

The bridge schema is canonically defined in `bridge/bridge_contract.schema.json` with SHA256-validated manifests, ensuring reproducible artifact generation and validation.

4 Implementation

4.1 AF_XDP Integration

The AF_XDP integration follows the standard UMEM/ring-buffer model [3, 4]. The Rust datapath creates a UMEM region with configurable frame sizes (default 4096 bytes) and chunk counts, registers it with the XDP socket, and maps the four ring buffers (RX, TX, FILL, COMPLETION) into process memory. The Go control plane prepares the host environment: setting IRQ affinity, allocating hugepages, and binding the XDP program to the target NIC queue.

A critical implementation detail is the `XDP_USE_NEED_WAKEUP` flag, which shifts the responsibility of triggering kernel ring consumption from the driver to the application, enabling batch-driven syscall amortization [5]. The Rust datapath batches packet descriptors and triggers `sendto/recvfrom` syscalls only when necessary, reducing system-call overhead.

4.2 Multi-Path Spraying (MRC)

Mohawk Nexus implements a Multi-path spraying mechanism (referred to as MRC in the codebase) that distributes packet flows across multiple paths with minimal overhead. Micro-benchmarks show spraying decisions at **18.1–20.1 ns/op** with **33–75 GB/s** effective throughput, adding almost zero overhead to the forwarding path [1]. The implementation achieves this through:

- Pre-computed path tables updated via batched control messages
- Lock-free spraying decisions using atomic indices
- Minimal allocations (1–3 allocs/op in the spraying hot path)

4.3 Cryptographic Engine

The crypto layer provides hybrid classical/post-quantum primitives with an emphasis on in-place operations to eliminate memory copies:

Table 1: Cryptographic Operation Performance

Operation	Latency	Allocations
EncryptInPlace	878–1,357 ns/op	1 alloc/op
DecryptInPlace	652–751 ns/op	0 allocs/op

These numbers compare favorably to WireGuard’s typical 1–2 Gbps/core for ChaCha20/Poly1305, though direct comparison is nuanced because Mohawk targets richer forwarding features and pluggable routing alongside crypto [1]. The zero-allocation decryption path is particularly notable for high-throughput scenarios.

4.4 Session and Routing

Session establishment is optimized through caching:

- **Cached session lookup:** 582–1,278 ns/op, 1376 B/op, 4 allocs/op
- **Uncached session creation:** 767–1,062 ns/op, 1392 B/op, 5 allocs/op

Route table updates are delivered from Go to Rust as batched deltas, avoiding per-packet routing table lookups across the language boundary. The Rust datapath uses BTree and hash-based indexes optimized for the forwarding hot path.

4.5 Toolchain and Reproducibility

The workspace pins Go to version 1.26.1 for reproducible builds and CI parity. A `.devcontainer/devcontainer.` provides a containerized development environment with the pinned toolchain pre-installed. The root `Makefile` exposes verification targets:

- `make verify` — root-level validation without unpublished subrepo dependencies
- `make verify-go` — Go unit tests and benchmarks
- `make verify-rust` — Rust datapath tests
- `make bridge-smoke` — end-to-end contract validation

Benchmark artifacts (logs, pprof CPU and memory profiles) are written to `SMIP-MWP/benchmarks/` and `benchmarks/baselines/`, enabling flamegraph analysis and regression tracking.

5 Performance Evaluation

5.1 Methodology

All reported micro-benchmarks were executed on AMD EPYC 64-core server-grade hardware with kernel-bypass optimizations enabled (hugepages, IRQ affinity, zero-copy AF_XDP where supported) [1]. Benchmarks use Go’s built-in benchmark framework (`-benchmem`) and Rust’s criterion-style harnesses, with pprof profiles captured for CPU and memory analysis.

5.2 Layer-by-Layer Results

Table 2: Full Stack Layer-by-Layer Performance Matrix

Layer	Operation	Mohawk Performance	Perfor-	Assessment
Rust Datapath (Hot Path)	Forwarding + Multi-path Spraying	2.4–2.5 Mpps (single core); up to 75 GB/s		Excellent for feature-rich path
Multi-Path Spraying (MRC)	Spraying + payload (4K/8K)	18.1–20.1 ns/op ; 33–75 GB/s		Outstanding
Crypto (Hybrid/PQC-ready)	In-place Encrypt / Decrypt	Encrypt: 878–1,357 ns/op ; Decrypt: 652–751 ns/op		Strong, especially PQC hybrid
Session & Routing	NewHybridSession (cached / uncached)	Cached: 1,278 ns/op ; Uncached: 767–1,062 ns/op		Good after optimizations
AF_XDP Integration	Packet RX/TX (kernel-bypass)	Tied to datapath (~2.4+ Mpps)		Competitive with full features
Full Stack (Combined)	End-to-end (spray + crypto + forward)	Estimated 1.8–2.3 Mpps sustained		Promising for sovereign/edge use

5.3 Stress Testing

Containerized stress runs use a dedicated `Dockerfile.stress` with privileged access to host networking. A 300-second high-load run with 32 concurrent operations and 12 benchmark iterations produced stable tail latencies and no memory leaks, as verified by pprof heap profiles. The stress harness is reproducible via:

Listing 4: Containerized Stress Test Invocation

```
docker build -f Dockerfile.stress -t smip-mwp:stress .
docker run --rm --privileged -v "$PWD":/app \
  -v "$PWD/benchmarks":/app/benchmarks \
  -w /app smip-mwp:stress \
  bash -lc 'DURATION=300 LOAD_LEVEL=high bash ./scripts/stress-test.sh'
```

5.4 Analysis

Mohawk Nexus deliberately trades raw maximum Mpps (where plain DPDK or simple AF_XDP forwarding excel) for a richer feature set: multi-path spraying, hybrid PQC crypto, pluggable

routing, and formal traceability. The 2.4+ Mpps figure represents a *feature-rich* datapath, not a minimal L2 forwarder. The low allocation counts (0–5 allocs/op across critical paths) indicate that the Rust implementation successfully avoids garbage-collection or allocator pressure that would degrade sustained throughput.

6 Verification and Traceability

6.1 Unified Traceability Matrix

The `TRACEABILITY_MATRIX.md` document connects sovereign protocol claims to implementation files across both languages [7]:

Table 3: Unified Traceability Matrix (Selected Concerns)

Concern	Sovereign Source	Go Binding	Rust Binding
Protocol semantics	FORMAL_TRACEABILITY_MATRIX.md	internal/bridge/contract.go	cli/src/bridge.rs
Session and key material	docs/SECURITY.md	internal/bridge/contract.go	cli/src/bridge.rs
Route and forwarding policy	docs/ARCHITECTURE.md	cmd/mohawk-node/main.go	cli/src/main.rs
AF_XDP runtime config	docs/PERFORMANCE.md	internal/datapath/afxdp/*	afxdp/*
Telemetry and health	docs/OPERATIONS.md	internal/bridge/contract.go	cli/src/bridge.rs

All concerns are validated through schema checks, semantic parity tests, bridge smoke paths, and workspace-level integration tests. The status across all mapped concerns is currently *aligned*.

6.2 Formal Artifacts

The `Sovereign-Mohawk-Proto` repository contains Lean 4 proof artifacts that model protocol semantics and security properties. While the current integration maps these proofs to the bridge contract rather than verifying the Rust code directly, the traceability matrix ensures that any change to the wire format, session model, or routing semantics is explicitly reviewed against the formal specification.

6.3 CI and Regression Prevention

GitHub Actions runs `make verify` from the repository root. A key design decision is that the root verify path is *resilient*—it does not depend on unpublished subrepo SHAs, allowing CI to remain green while component pull requests are in flight. This is achieved through root-owned shims and fallback stubs that stand in for missing subrepo code during integration periods.

7 Unification Methodology

The Mohawk Nexus repository was not created from scratch; it represents the unification of three pre-existing repositories (`Sovereign-Mohawk-Proto`, `SMIP-MWP`, `SMIP-MWP-Rust`) into a single workspace. The `UNIFICATION_PLAN.md` document prescribes a six-phase integration strategy [8]:

Phase 1: Define the Contract. Freeze the canonical packet format, session model, and routing semantics; write a shared interface spec for configuration, route updates, session setup, and health reporting.

Phase 2: Carve by Responsibility. Move policy and verification artifacts into a `spec/` layer; keep Go as the integration surface and Rust as the execution surface; avoid duplicating packet parsing or crypto decisions.

Phase 3: Consolidate the Datapath. Make Rust the single packet-processing engine; expose a stable boundary for route updates, key material, and counters; replace per-packet cross-language calls with batched control messages.

Phase 4: Rebuild Orchestration. Let Go own startup, config loading, cluster management, telemetry, and test harnesses; deliver control-plane state to Rust in batches.

Phase 5: Reattach Verification. Map sovereign protocol proofs to the unified packet and session model; add tests proving the implementation matches the documented contract; keep security-sensitive decisions traceable to a spec or proof artifact.

Phase 6: Performance Validation. Establish a single benchmark baseline before moving code; compare the unified stack against separate Go and Rust baselines; fail the merge if per-packet allocation increases or language-boundary overhead is added.

7.1 Acceptance Criteria

The unification is considered successful only if the final stack demonstrates:

- One authoritative protocol contract
- One datapath implementation
- One control-plane/orchestration layer
- Preserved or improved throughput versus the current Rust path
- No regressions in verification or security traceability
- Reproducible benchmarks and deployment instructions

8 Discussion

8.1 Strengths

1. **Safety at Speed:** By implementing the hot path in Rust, Mohawk Nexus achieves near-DPDK performance while eliminating entire classes of memory-safety vulnerabilities (use-after-free, buffer overflows) that plague C/C++ datapaths.
2. **Operational Simplicity:** The Go control plane provides a familiar, ecosystem-rich environment for operators, with standard tooling for metrics, logging, and container orchestration.
3. **Sovereign Cryptography:** The hybrid PQC-ready crypto layer prepares the stack for post-quantum transitions without requiring a full rewrite.
4. **Reproducibility:** Pinned toolchains, containerized stress tests, and SHA256-validated bridge manifests ensure that builds and benchmarks are reproducible across environments.

8.2 Limitations and Future Work

1. **AF_XDP Driver Support:** Zero-copy mode requires explicit NIC and driver support. Fallback to copy mode incurs performance penalties. Expanding the tested hardware matrix is ongoing work.

2. **Formal Verification Gap:** Current formal artifacts model protocol semantics but do not yet verify the Rust implementation directly. Future work includes integrating Rust verification tools (e.g., Kani, Creusot) or extracting verified code from Lean.
3. **Single UMEM Scalability:** While `XDP_SHARED_UMEM` allows socket sharing, the single-producer/single-consumer `FILL` and `COMPLETION` rings require careful synchronization when scaling across multiple queues and NUMA nodes.
4. **Control Plane Granularity:** The current batched update model is efficient but may introduce latency for time-sensitive route changes. Investigating incremental delta encoding or priority-batched updates is a potential optimization.

8.3 Comparison to Existing Stacks

vs. DPDK: DPDK achieves higher raw Mpps for simple forwarding but requires dedicated cores, custom drivers, and sacrifices kernel integration. Mohawk Nexus targets richer L3+ features with lower operational overhead.

vs. WireGuard: WireGuard is optimized for single-packet crypto VPN paths. Mohawk Nexus integrates crypto with multi-path spraying and pluggable routing, targeting federation and edge-computing scenarios rather than point-to-point tunnels.

vs. Standard Kernel Stack: The Linux kernel stack handles general-purpose protocols robustly but cannot match the per-packet latency and allocation profiles of a user-space `AF_XDP` datapath.

9 Conclusion

Mohawk Nexus presents a reference architecture for building high-performance, safe, and formally traceable network stacks in the Linux ecosystem. Its deliberate separation of Go orchestration from Rust packet processing—governed by a canonical bridge contract—demonstrates that multi-language integration need not sacrifice performance. The integration of `AF_XDP` kernel-bypass, multi-path spraying, hybrid post-quantum cryptography, and formal verification traceability positions the stack for sovereign networking scenarios where both speed and assurance are non-negotiable.

The six-phase unification methodology and reproducible benchmark harnesses provide a template for similar consolidation efforts, while the traceability matrix offers a concrete mechanism for keeping protocol proofs and implementation code aligned. As network infrastructure confronts post-quantum transitions, edge federation, and increasingly adversarial threat models, designs such as Mohawk Nexus illustrate a viable path forward.

References

- [1] R. Williams et al., “Mohawk Nexus Unified Workspace,” GitHub Repository, 2026. <https://github.com/rwilliamsbg-ops/Mohawk-Nexus>
- [2] J. Corbet, “Accelerating networking with `AF_XDP`,” *Linux Weekly News*, Apr. 2018. <https://lwn.net/Articles/750845/>
- [3] “`AF_XDP`,” *The Linux Kernel Documentation*, v6.1. https://www.kernel.org/doc/html/v6.1/networking/af_xdp.html
- [4] M. G. Blog, “Recapitulating `AF_XDP`,” *Medium*, Mar. 2022. <https://medium.com/high-performance-network-programming/recapitulating-af-xdp-ef6c1ebad8>

-
- [5] “AF_XDP 101: Getting Started With an In-Kernel Network Stack Bypass,” *Emumba Blog*, Jun. 2020. <https://blog.emumba.com/af-xdp-101-getting-started-with-an-in-kernel-network-stack-bypass-6e6901a285c8>
 - [6] R. Williams et al., “Go-Rust Bridge Contract,” `BRIDGE_CONTRACT.md`, Mohawk-Nexus Repository, 2026.
 - [7] R. Williams et al., “Unified Traceability Matrix,” `TRACEABILITY_MATRIX.md`, Mohawk-Nexus Repository, 2026.
 - [8] R. Williams et al., “Mohawk Network Stack Unification Plan,” `UNIFICATION_PLAN.md`, Mohawk-Nexus Repository, 2026.
 - [9] Intel DPDK Documentation, “DPDK Getting Started Guide.” https://doc.dpdk.org/guides/linux_gsg/
 - [10] “XDP (eXpress Data Path) as a building block for other...” *FOSDEM 2019*. https://archive.fosdem.org/2019/schedule/event/xdp_overview_and_update/attachments/slides/2877/export/events/attachments/xdp_overview_and_update/slides/2877/xdp_building_block.pdf
 - [11] Cloudflare Blog, “A debugging story: corrupt packets in AF_XDP,” Jan. 2023. <https://blog.cloudflare.com/a-debugging-story-corrupt-packets-in-af-xdp-kernel-bug-or-user-error/>